Lecture 7 — BOM in JavaScript

🌐 **BOM (Browser Object Model)** is everything the browser exposes to JavaScript outside the page content itself. It lets JS interact with the browser window, URL, history, storage, dialogs, timers, and more.

1) BOM vs DOM

- **DOM**: the *document* (HTML) as a tree of nodes. Example: `document.querySelector()`
- **BOM**: the *browser environment* around the document. Example: `window.location`, `history`, `localStorage`

2) The `window` object

- In browsers, `window` **is the global object**.
- Many globals are properties of `window`.
  - `alert()` is `window.alert()`
  - `setTimeout()` is `window.setTimeout()`

```
console.log(window === globalThis); // true (in browsers)
```

3) `location` (URL controls)

- Read parts of the current URL.
- Redirect or reload.

```
console.log(location.href);       // full URL
console.log(location.hostname);   // domain
console.log(location.pathname);   // path
console.log(location.search);     // query string
```

```
// redirect
location.href = "https://example.com";

// reload
location.reload();
```

4) `history` (back/forward)

```
history.back();
history.forward();
history.go(-2);
```

5) `navigator` (browser + device info)

```
console.log(navigator.userAgent);
console.log(navigator.language);
console.log(navigator.onLine);
```

6) Screen and window size

```
console.log(screen.width, screen.height);

console.log(window.innerWidth, window.innerHeight); // viewpo
rt
console.log(window.outerWidth, window.outerHeight); // browse
r window

window.addEventListener("resize", () => {
    console.log(innerWidth, innerHeight);
});
```

7) Dialogs

```
alert("Hi");
const ok = confirm("Continue?");
const name = prompt("Your name?");
```

8) Timers (`setTimeout`, `setInterval`)

- `setTimeout(fn, delayMs)`: runs **once** after a delay.
- `setInterval(fn, delayMs)`: runs **repeatedly** every delay.
- Both return an ID you can cancel.

```
// setTimeout: runs once
const timeoutId = setTimeout(() => {
    console.log("Runs once after 1 second");
}, 1000);

clearTimeout(timeoutId);

// setInterval: runs repeatedly
const intervalId = setInterval(() => {
    console.log("Runs every 2 seconds");
}, 2000);

clearInterval(intervalId);

// common pattern: stop after N runs
let count = 0;
const id = setInterval(() => {
    count++;
    console.log(count);
    if (count === 5) clearInterval(id);
}, 1000);
```

9) Web Storage

- `localStorage`: persists until manually cleared.
- `sessionStorage`: cleared when the tab is closed.

```
localStorage.setItem("theme", "dark");
console.log(localStorage.getItem("theme"));
localStorage.removeItem("theme");

sessionStorage.setItem("token", "123");
```

10) Cookies (quick note)

- Older storage mechanism, sent with HTTP requests.
- Can be set via `document.cookie`.

```
document.cookie = "lang=en; max-age=3600";
console.log(document.cookie);
```

Note: `innerWidth` and `innerHeight`

- `window.innerWidth`: the **viewport width** in CSS pixels (the visible area inside the tab, excluding browser UI).
- `window.innerHeight`: the **viewport height** in CSS pixels.
- Useful for responsive logic, and they change on resize and sometimes on mobile when the address bar shows or hides.

```
console.log(window.innerWidth, window.innerHeight);
```

Everything intialized in parent(window)

```
var x = 90
console.log(window.x) // 90
```

Open a new tab when clicking

```
<button id='openFace'>Open Facebook</button>
```

```
var btn = document.getElementById('openFace')

function openFacebook(){
// window.open(target_link, title, "new tab size/dimensions")
    window.open("https://www.facebook.com", "title1", "width=
600,height=600")
}

btn.onclick = openFacebook
```

When focusing on the main or index.html, the window goes down rather than gets deleted

```
<button id='closeFace'>Close Facebook</button>
```

```
var btn1 = document.getElementById('openFace')
var btn2 = document.getElementById('closeFace')
let x;
function openFacebook(){
// window.open(target_link, title, "new tab size/dimensions")
    x = window.open("https://www.facebook.com", "title1", "wi
dth=600,height=600")
}
btn1.onclick  = openFacebook
function closeFacebook(){
    x.close()
}
btn2.onclick = closeFacebook
```

```
console.log(window.screen.width) // same1
console.log(screen.width)//same1
```

```
console.log(screen.availWidth)
console.log(window.innerWidth) // if there a side bar the tot
al width of screen is subtracted.
```

```
console.log(screen.height) //laptop height
console.log(screen.availheight) // labtop height - nav bar yo
u have
console.log(window.innerHeight) // it get effected with anyth
ing
//like availheight + sidebar_height
```

when user click “buy now” move to payment method page

```
location.assign('payment\index.html')
```

when user “login” reload the page

```
location.reload()
```

when you want the reference of page

```
location.href
```

History

- `history.back()` goes to the previous page in the session history (same as the browser back button).
- `history.forward()` goes to the next page (same as the forward button).
- There is **no** `history.backward()` method.

```
history.back();
history.forward();

// or jump by N steps
```

```
history.go(-1); // back
history.go(1);  // forward
```

⇒ We can say it's the arrow that appears in the top-left corner of the page

```
// these functions are deprecated aren't used today
console.log(navigator.appName)
console.log(navigator.appCodeName)
console.log(navigator.appVersion)
console.log(navigator.platform)
```

```
console.log(navigator.language) => default language
console.log(navigator.online) =>
```

"navigator.online" ⇒ you can use it to calculate how much user is online

Time

```
// for only one time
setTimeout(function, time)
// for infinite
setInterval()
```

```
var time1 = setTimeout( function(){
alert("please sign in")
}, 2000)
```

or some malicious website

```
var add = setTimeout(function(){
location.assign("adds.html")
},1000)
```

Every 1s print "ahmed"

```
var time2 = setInterval(function(){
console.log("ahmed")
}, 1000)
```

when to stop function

```
clearTimeout(time1)
```

you put it inside a function